



Instituto Tecnológico de Mazatlán

Semáforo Vehicular y Peatonal con Máquina de Estados Finitos

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Semáforo Vehicular y Peatonal con Máquina de Estados Finitos

1. Descripción del proyecto

Esta práctica consiste en desarrollar un sistema de semáforo vehicular y peatonal utilizando una máquina de estados finitos (FSM) programada en Arduino UNO R4 WiFi. El sistema alterna automáticamente entre los estados de un semáforo convencional (verde, amarillo y rojo vehicular) e incorpora un botón peatonal que permite solicitar el cruce, atendiendo la petición de forma segura sin interrumpir abruptamente el ciclo vehicular.

Toda la temporización del sistema se maneja con `millis()`, sin utilizar `delay()` en ningún punto del programa, lo que permite que la lectura del botón y la lógica de estados se evalúen de forma continua y sin bloqueos.

2. Objetivos

- Modelar un sistema secuencial como una máquina de estados finitos (FSM) utilizando `enum class`.
- Implementar transiciones de estado controladas por tiempo (`millis()`) y por eventos externos (botón).
- Aplicar una técnica de antirrebote (debounce) por software para la lectura confiable de un botón.
- Diseñar reglas de prioridad y seguridad: la solicitud peatonal solo se atiende en el momento adecuado del ciclo.
- Reforzar el uso de funciones para separar responsabilidades: aplicar salidas, cambiar de estado y leer el botón.

3. Herramientas y material utilizado

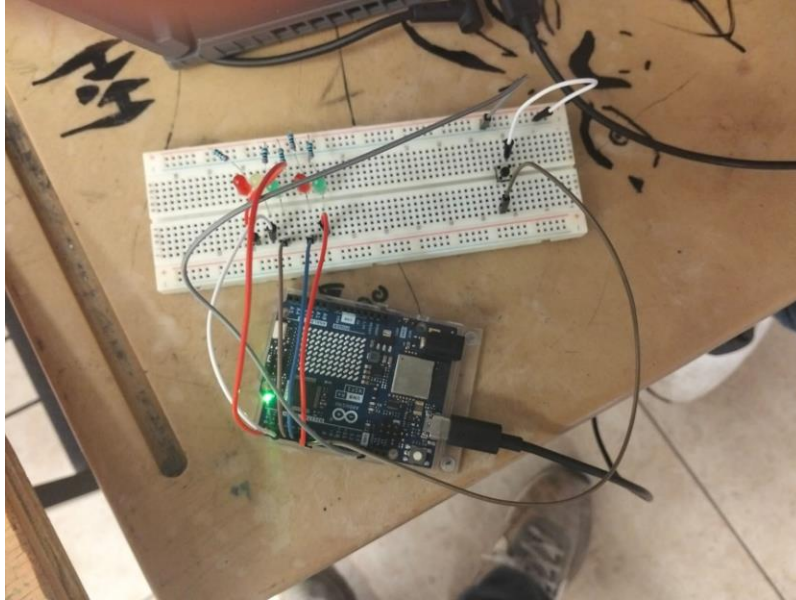
- Arduino UNO R4 WiFi
- Arduino IDE
- Protoboard
- LEDs: rojo, amarillo y verde (vehicular) y rojo/verde (peatonal)
- Resistencias limitadoras de corriente para cada LED
- Pulsador (botón) para la solicitud peatonal
- Cables de conexión (jumpers)

4. Diagrama

El diagrama representa la conexión utilizada para los cinco LEDs y el pulsador del sistema:

- `PIN_VEH_ROJO` → Pin digital 2
- `PIN_VEH_AMARILLO` → Pin digital 3
- `PIN_VEH_VERDE` → Pin digital 4
- `PIN_PEA_ROJO` → Pin digital 5
- `PIN_PEA_VERDE` → Pin digital 6
- `PIN_BOTON` → Pin digital 7 (configurado como `INPUT_PULLUP`)

Cada LED se conecta en serie con su resistencia limitadora hacia GND, compartiendo un mismo riel de tierra junto con el pulsador. A continuación se muestra el montaje físico utilizado durante la práctica:



5. Metodología

Primero se armó el circuito físico con los cinco LEDs (tres vehiculares y dos peatonales) y el pulsador, conectados a los pines 2 al 7 del Arduino UNO R4 WiFi. Con el montaje listo, se modeló el comportamiento del semáforo como una máquina de estados finitos, definiendo cuatro estados posibles mediante un enum class: VEH_VERDE, VEH_AMARILLO, VEH_ROJO y PEA_VERDE.

Se programó una función encargada de aplicar las salidas correspondientes a cada estado (apagando todos los LEDs y encendiendo únicamente los que pertenecen al estado actual) y otra función encargada de realizar la transición, registrando el instante de entrada a cada estado con `millis()`. Posteriormente se implementó la lectura del pulsador con un filtro de antirrebote basado en dos variables: la última lectura cruda del pin y el último estado ya confirmado como estable, reportando una pulsación válida únicamente en el flanco de bajada.

Finalmente, se definieron las reglas de prioridad: la solicitud peatonal solo se arma si el botón se presiona durante los estados VEH_VERDE o VEH_AMARILLO, y se atiende en cuanto el semáforo vehicular llega a VEH_ROJO. Se realizaron pruebas presionando el botón en distintos momentos del ciclo para verificar que estas reglas se cumplieran correctamente.

6. Explicación del funcionamiento

6.1 Máquina de estados finitos (FSM)

El sistema se modela con un enum class llamado `EstadoSemaforo`, con cuatro estados posibles. El uso de enum class evita fugas de nombres al espacio global y obliga a un tipado fuerte, reduciendo errores al comparar estados.

```
void cambiarEstado(EstadoSemaforo nuevoEstado) {  
    estadoActual = nuevoEstado;  
    inicioEstado = millis();  
    aplicarSalidasEstado(nuevoEstado);  
}
```

Cada vez que el sistema cambia de estado, se registra el tiempo de entrada y se actualizan las salidas correspondientes, evitando estados intermedios inconsistentes.

6.2 Antirrebote del botón (debounce)

La función `botonPresionado()` separa la última lectura cruda del pin (usada solo para detectar cambios) del estado ya confirmado como estable. Solo cuando la lectura permanece estable durante más de 50 ms se considera un cambio real, y únicamente se reporta una pulsación válida en el flanco de bajada, evitando falsos positivos por rebote mecánico del botón.

6.3 Reglas de la solicitud peatonal

La solicitud de cruce peatonal solo se "arma" si el botón se presiona mientras el semáforo vehicular está en VEH_VERDE o VEH_AMARILLO. Si se presiona en otro momento, se ignora. Una vez armada, la solicitud se atiende en cuanto el semáforo vehicular llega a VEH_ROJO, transicionando de inmediato a PEA_VERDE. Si nadie solicita el cruce, el ciclo vehicular continúa por sí solo una vez transcurrido su tiempo en rojo (6000 ms).

7. Análisis de resultados

Durante las pruebas se verificó que cada estado de la FSM activa exactamente las salidas que le corresponden: en VEH_VERDE se enciende el LED verde vehicular junto con el rojo peatonal; en VEH_AMARILLO, el amarillo vehicular con el rojo peatonal; en VEH_ROJO, el rojo vehicular con el rojo peatonal; y en PEA_VERDE, el verde peatonal mientras el vehicular permanece en rojo. No se observaron combinaciones inválidas de LEDs encendidos en ningún momento.

Se probó el botón en distintos momentos del ciclo: al presionarlo durante VEH_VERDE o VEH_AMARILLO, la solicitud se armó correctamente y fue atendida en cuanto el semáforo llegó a VEH_ROJO. Al presionarlo durante VEH_ROJO o PEA_VERDE, la pulsación se ignoró, tal como se esperaba según las reglas de prioridad diseñadas. Cuando no se presionó el botón, el ciclo vehicular (verde → amarillo → rojo) se repitió de forma consistente e indefinida.

También se verificó el funcionamiento del antirrebote presionando el botón de forma rápida y repetida: el sistema registró una sola pulsación válida por cada presión física, confirmando que la lógica de 50 ms evitó activaciones falsas de la solicitud peatonal. En general, el sistema se comportó de manera predecible y estable durante toda la prueba, validando tanto la lógica de programación como el montaje físico del circuito.

8. Conclusiones

Gabriel

En esta práctica comprendí cómo modelar un sistema de control secuencial como una máquina de estados finitos. Ver cómo cada estado tiene su propia acción de entrada y su propia condición de salida me ayudó a entender por qué este patrón se usa tanto en sistemas reales: hace que el comportamiento sea predecible y fácil de seguir, en lugar de tener una maraña de condicionales sueltos.

Javier

Durante esta práctica aprendí a implementar un filtro de antirrebote por software, separando la lectura cruda del pin de un estado ya confirmado como estable. También me quedó clara la importancia de definir reglas de prioridad explícitas, como decidir en qué momentos del ciclo se acepta o se ignora la solicitud del botón, para que el sistema se comporte de forma segura y predecible.

Rosa

Esta práctica me permitió comprender cómo combinar temporización no bloqueante con eventos externos asíncronos dentro de una misma máquina de estados. Ver que el semáforo seguía funcionando de forma correcta sin importar en qué momento se presionara el botón, gracias a las reglas de prioridad y al antirrebote, me hizo entender la diferencia entre programar solo para el caso esperado y programar pensando en todos los casos posibles.